Entrega 2 - Documento de diseño

Cambios respecto a la primera entrega:
Respecto al UML presentado en la primera entrega, para esta entrega se realizaron modificaciones con el fin de mejorar la implementación y el funcionamiento del sistema. Los cambios fueron: agregación de 5 clases, se convirtió Categoria en enum, se sacaron las sanciones de su propia clase y se movieron las listas del club desde Administrados hacía ClubSeneca.

- **Una clase principal/punto de entrada para todo el sistema:** Se agregó la clase ClubSeneca con el fin de ser la clase madre donde se busca guardar información del club y redirigir las operaciones que el usuario dentro del sistema desea realizar. Junto a esta clase se agregaron 4 clases más que sirven como apoyo para el inicio de sesión, la persistencia, los reportes y los turnos del personal.
- **Los datos ya no son del administrador:** Previamente, se mencionaba como era el administrador el encargado de guardar socios,equipos, sugerencias y solicitudes, ahora esa información pertenece al club y es el administrador el que ejecuta acciones sobre ellas
- **La categoría pasó de ser clase a enum:** Ahora son 4 valores fijos que no cambian de comportamiento
- **El equipo ya no se guarda en socios directamente:** Equipo guarda inscripciones ya que los valores de la posición, el número de camiseta y las sanciones pertenecen a la relación del jugador con el equipo más no de una de estas dos clases.
- **Las sanciones tienen ahora su propia clase:** Ahora las sanciones y las operaciones de tarjetas se usa en el club y en cada competencia
- **Se completaron conflictos que no se habían considerado anteriormente:** Se encontró un problema que anteriormente no se validaba que un partido interno sea compatible entre equipos, adicionalmente que nadie pudiese entrar a una cancha techada con una bebida caliente y se informa sobre los alérgenos antes de vender un snack
- **El partido de práctica se separó del partido de conjunto:** Ahora funciona con jugadores individuales, otras estadísticas y un ranking
- **Todas las clases recibieron su constructor y métodos de consulta como debe de ser enun modelo de diseño**
- **Restricciones nuevas:** agregamos R23 (mínimo de 2 entrenadores y 1 fisioterapeuta en los turnos) y R24 (mínimo de jugadores habilitados para jugar un partido interno), porque en la primera entrega esas dos reglas no tenían una restricción propia.

Supuestos: Para la realización del diseño del sistema se tuvo que tomar supuestos en ciertos elementos con el fin de simplificar ciertas funcionalidades de nuestro sistema y la interacción de ciertos elementos dentro de este con el fin de cumplir con los requisitos funcionales y que el sistema pueda operar de manera propia y adecuada.

- Se estableció rangos de edad con lo que se puede determinar la categoría de cada jugador dependiendo del rango de edad en el que se encuentre

- El socio pertenece a un único equipo por deporte pero se puede encontrar en equipos de otros deportes a la vez
- La cantidad mínima de jugadores en cada deporte de conjunto
- El tiempo establecido que debe esperar una persona que ordenó una bebida caliente para hacer uso de una instalación techada, sin embargo este tiempo puede cambiar
- En voleibol el marcador local y visitante cuenta sets ganados, no se guarda los puntos de estos
- El código de descuento no es acumulable con otras promociones
- El periodo de los reportes permiten únicamente “diario”, “semanal”, y “mensual”
- Para el ELO se estableció como punto de partida el valor 1000 así como se hace en ajedrez y partir de ello sube o baja su valor dependiendo del rendimiento del socio
- El socio se encuentra ocupando una instalación si este tiene una reserva en curso en ella
- Los partidos de práctica se guardan aparte en Historial porque PartidoPractica no hereda de Partido

Estereotipos

Estereotipo	**Candidato**
Information Holder: Solo guarda y entrega datos. Tiene solo get y set	Usuario, Socio, Empleado, Entrenador, Fisioterapeuta, Categoria, Administrador, InscripcionEquipo, RegistroNomina, InscripcionCompetencia, InscripcionIndividual, Reserva, Turno, SolicitudCambioTurno, AsistenciaEntrenamiento, EstadisticaConjunto y subclases, EstadisticaPractica y subclases, PartidoExterno, Producto, Articulo, Bebida, Snack, LineaVenta, CodigoDescuento, SugerenciaArticulo, Ubicacion, Intalación y subclases, RankingJugadorDisciplina, EstadoJugador, FaseASCUN, Modalidad, EstadoAsistencia, Alergeno, Rubro, EstadoSugerencia, EstadoSolicitud
Structurer: Guarda una colección y aplica reglas sobre ella	Equipo, Partido, Competencia, TorneoCerros, ASCUN, DisciplinaIndividual, SesionEntrenamiento, Tienda, CafeteriaSocios, Inventario, Historial, ClubSeneca
Service provider: Hace un cálculo o servicio puntual cuando se lo piden, sin mantener estado propio relevante	NivelJugador, DeporteConjunto y sublcases, ControlSanciones, Autenticador, GeneradorReportes
Controller: Toma decisiones y orquesta un flujo pidiéndoles trabajo a otros. Decide algo que involucra a varios objetos	PartidoPractica, PartidoInterno, Venta, GestorTurnos
Coordinator: Recibe peticiones y las reparte, sin decidir las reglas de	ClubSeneca

negocio	
Interfacer: Habla con algo externo al sistema (archivos, consola, usuario)	Persistencia, programas de prueba por consola

Decisiones de control

- **ClubSeneca es el único coordinador.** Crea y guarda las colecciones principales, recibe las acciones de los programas de prueba y las pasa a quien decide.
- **Administrador es un information holder.** Es un Usuario con permisos: valida que la acción sea suya y le pasa la petición a ClubSeneca. Así evitamos que se vuelva una clase que lo hace todo.
- **Cuatro controladores, uno por flujo importante:** Venta (compras), PartidoInterno (partidos internos), PartidoPractica (partidos de práctica) y GestorTurnos (cambios de turno). Cada flujo tiene un solo lugar donde se decide.
- **ControlSanciones es una clase aparte** porque el club y cada competencia llevan su propio conteo de amarillas con las mismas reglas.

Descripción de cada objeto, por grupos

Grupo 1: Usuarios

Objeto	**Estereotipo**	**Descripcion**
Usuario	Information Holder	Persona con acceso al sistema, guarda nombre, login y password
Socio	Information Holder	Miembro del club y jugador; guarda edad, puntos de fidelidad y cuándo recibió una bebida caliente
Empleado	Information Holder	Socio que trabaja en el club; guarda su código de empleado, su código de descuento y sus turnos
Entrenador, Fisioterapeuta	Information Holder	Tipos de empleado con sus funciones laborales propias
Administrador	Information Holder	Usuario con permisos administrativos que no es socio, solo valida permisos y pasa la petición a ClubSeneca

Grupo 2: Deportes de conjunto y competencias

Objeto	**Estereotipo**	**Descripción**
DeporteConjunto y Futbol, Voleibol, Baloncesto	Service provider	Reglas propias de cada deporte: mínimo de jugadores y puntos por resultado
Categoria (enum)	Information holder	Cuatro valores fijos (SUB13, SUB15, SUB17, MAYORES) con su rango de edad,

		verifica si una edad cabe
Equipo	Structurer	Agrupa a los jugadores de un deporte y categoría, controla cupo, camiseta única y habilitados
InscripcionEquipo	Information holder	Vincula un socio con un equipo: posición, número de camiseta y su ControlSanciones
ControlSanciones	Service provider	Lleva las amarillas acumuladas y el estado de un jugador, y decide si queda suspendido (2 amarillas o 1 roja)
Competencia	Structurer	Circuito externo: valida requisitos, inscribe equipos, guarda partidos y arma clasificaciones
TorneoCerros	Structurer	Liga de todos contra todos, mantiene la tabla de posiciones
ASCUN	Structurer	Competencia por fases (grupos y eliminación directa)
InscripcionCompetencia	Information holder	Registro de un equipo en una competencia y su nómina
RegistroNomina	Information holder	Jugador de la nómina de una competencia, con su propio ControlSanciones

Grupo 3: Partidos de equipo y estadísticas

Objeto	**Estereotipo**	**Descripción**
Partido	Structurer	Enfrentamiento con fecha y lista de estadísticas
PartidoInterno	Controller	Partido entre dos equipos del club, decide si se juega, registra el marcador y determina el ganador
EstadisticaConjunto y EstadisticaFutbol, EstadisticaBaloncesto, EstadisticaVoleibol	Information holder	Cifras individuales de un jugador en un partido, según el deporte (en fútbol también las tarjetas)
PartidoExterno	Information holder	Partido contra una institución rival: marcador, rival, escenario y el equipo del club que jugó

Grupo 4: Disciplinas individuales

Objeto	**Estereotipo**	**Descripción**

DisciplinaIndividual y Tenis, Padel, TenisDeMesa	Structurer	Academia de raqueta con su máximo de instalaciones simultáneas, decide si hay cupo
InscripcionIndividual	Information holder	Vincula un socio con una disciplina y su nivel
NivelJugador (enum)	Service provider	Principiante, Intermedio o Avanzado, decide si dos niveles son compatibles
Reserva	Information holder	Bloque horario de una instalación para un socio, sabe si se cruza con otro
PartidoPractica	Controller	Partido entre socios de una disciplina, valida niveles, define el ganador y dispara estadísticas y ranking
EstadisticaPractica y EstadisticaRaqueta, EstadisticaTenisMesa	Information holder	Cifras de un jugador en un partido de práctica
RankingJugadorDisciplina	Information holder	Puntaje ELO de un jugador en una disciplina, sabe actualizarlo

Grupo 5: Instalaciones

Objeto	**Estereotipo**	**Descripción**
Instalacion	Information holder	Espacio del club con nombre y capacidad
InstalacionIndividual y CanchaTenis, CanchaPadel, MesaTenisDeMesa	Information holder	Espacio de una disciplina individual, sabe si es techado, si admite una modalidad y si deja usarlo a quien lleva una bebida caliente
InstalacionConjunto y CanchaFutbol, CanchaBaloncesto, CanchaVoleibol	Information holder	Espacio para deportes de conjunto, existe para poder asignar sesiones de entrenamiento

Grupo 6: Entrenamiento

Objeto	**Estereotipo**	**Descripción**
SesionEntrenamiento	Structurer	Sesión planeada con fecha, hora, duración, instalación, convocados y asistencias
AsistenciaEntrenamiento	Information holder	Asistencia de un convocado, con su evaluación y comentario

Grupo 7: Tienda, cafetería e inventario

Objeto	Estereotipo	Descripción
Tienda	Structurer	Catálogo de artículos; verifica existencias y crea la Venta con IVA
CafeteriaSocios	Structurer	Catálogo de bebidas y snacks, verifica la regla de bebida caliente y crea la Venta con impuesto al consumo
Producto y Articulo, Bebida, Snack	Information holder	Algo que se vende, con nombre, categoría, precio y cantidad (bebida: caliente, snack: alérgenos)
Venta	Controller	Transacción con fecha y rubro, calcula el total con impuestos y descuento, y sus puntos
LineaVenta	Information holder	Producto y cantidad dentro de una venta
CodigoDescuento	Information holder	Código del empleado: 15% para él y 8% para quien lo comparta
Inventario	Structurer	Reúne artículos, instalaciones y ubicaciones, mueve artículos entre ubicaciones
Ubicacion	Information holder	Lugar de almacenamiento, con los artículos que tiene
SugerenciaArticulo	Information holder	Propuesta de un empleado con su estado (nace pendiente)

Grupo 8: Turnos de empleados

Objeto	Estereotipo	Descripción
Turno	Information holder	Franja semanal de un empleado, sabe si cubre un día y hora
SolicitudCambioTurno	Information holder	Solicitud con el solicitante, el turno actual, el turno nuevo y su estado (nace pendiente)
GestorTurnos	Controller	Decide si una solicitud se puede aprobar sin bajar de 2 entrenadores y 1 fisioterapeuta

Grupo 9: Núcleo del sistema

Objeto	Estereotipo	Descripción
Historial	Structurer	Reúne reservas, partidos y sesiones para consulta del administrador

ClubSeneca	Coordinator y structurer	Punto de entrada: crea y sostiene las colecciones raíz y los componentes, y delega cada petición
Autenticador	Service provider	Verifica login y password
GeneradorReportes	Service provider	Arma reportes de ventas por rubro y periodo, y reportes deportivos
Persistencia	Interfacer	Guarda y carga los datos en archivos

Responsabilidades

#	Responsabilidad	Componente (quien reciba la petición y la reparte)	Decide (la clase donde vive la regla)
1	Inscribir jugador (cupo, camiseta única, un equipo por deporte, categoría)	ClubSeneca.inscribirJugador()	Equipo.agregarJugador(), Equipo.camisetaOcupada(), Socio.tieneEquipoEn(), Categoria.admiteEdad()
2	Registrar tarjetas y suspender	ClubSeneca.registrarPartidoInterno()	InscripcionEquipo.registrarTarjetas(), ControlSanciones.registrarTarjetas(), ControlSanciones.suspender()
3	Decidir si un partido puede jugarse	ClubSeneca.registrarPartidoInterno()	PartidoInterno.puedeJugarse(), Equipo.puedeJugar()
4	Registrar marcador y ganador	ClubSeneca.registrarPartidoInterno()	PartidoInterno.registrarMarcador(), PartidoInterno.getGanador()
5	Puntos de liga y tabla	ClubSeneca.registrarPartidoExterno()	DeporteConjunto.calcularPuntos(), TorneoCerros.actualizarTabla()
6	Validar y registrar equipo en competencia	ClubSeneca.inscribirEquipoEnCompetencia( )	Competencia.equipoCumpleRequisitos(), Competencia.inscribirEquipo(), InscripcionCompetencia.agregarANomina()
7	Iniciar esa inscripción (RF29)	Administrador.inscribirEquipoEnCompetencia() valida el permiso y llama a ClubSeneca.inscribirEquipoEnCompetencia()	Competencia.equipoCumpleRequisitos(), Competencia.inscribirEquipo(), InscripcionCompetencia.agregarANomina()

8	Registrar partido y estadísticas (RF16)	ClubSeneca.registrarPartidoInterno(), ClubSeneca.registrarPartidoExterno()	Partido.agregarEstadistica()
9	Validar cupo de reserva	ClubSeneca.reservar()	DisciplinaIndividual.hayCupo(), InstalacionIndividual.admiteModalidad(), Reserva.seSolapaCon()
10	Regla de la bebida caliente en las instalaciones (R21)	ClubSeneca.reservar()	InstalacionIndividual.puedeUsar(), Socio.llevaBebidaCaliente()
11	Crear y guardar la reserva	ClubSeneca.reservar()	Historial.agregarReserva()
12	Validar niveles de un partido de práctica	ClubSeneca.programarPartidoPractica()	PartidoPractica.validarNiveles(), NivelJugador.esCompatibleCon(), Socio.getNivelEn(), Socio.esEntrenador()
13	Actualizar el ELO	ClubSeneca.finalizarPartidoPractica()	RankingJugadorDisciplina.actualizarPuntaje()
14	Encontrar el ranking de un jugador en una disciplina	ClubSeneca.getRanking()	no aplica
15	Planear una sesión y convocar	ClubSeneca.planearSesion()	SesionEntrenamiento.agregarConvocado()
16	Revisar que la instalación esté libre para la sesión (H8)	ClubSeneca.planearSesion()	SesionEntrenamiento.seSolapaCon()
17	Registrar asistencia solo de convocados (H9)	ClubSeneca.registrarAsistencia()	SesionEntrenamiento.registrarAsistencia()
18	Registrar la evaluación de 1 a 5	ClubSeneca.registrarAsistencia()	AsistenciaEntrenamiento.registrarEvaluacion()
19	Crear la venta de tienda o cafetería	ClubSeneca.registrarVenta()	Tienda.vender(), CafeteriaSocios.vender(), CafeteriaSocios.puedeDespachar()

20	Calcular el total con impuesto, descuento y puntos	ClubSeneca.registrarVenta()	Venta.calcularTotal(), CodigoDescuento.getPorcentaje()
21	Sumar el 2% de puntos y redimirlos	Venta lo hace al calcular el total	Socio.acumularPuntos(), Socio.redimirPuntos()
22	Controlar las existencias	las pide CafeteriaSocios o Tienda	Producto.reducirCantidad(), Producto.agregarCantidad()
23	Guardar todas las ventas (RF27)	ClubSeneca (lista de ventas)	no aplica
24	Mover artículos entre ubicaciones	ClubSeneca.moverArticulo()	Inventario.moverArticulo(), Ubicacion.tieneArticulo()
25	Reabastecer la tienda	ClubSeneca.reabastecerTienda()	Tienda.reabastecer()
26	Crear sugerencias (H6)	ClubSeneca.registrarSugerencia()	Empleado.sugerirArticulo()
27	Aprobar o rechazar sugerencias	Administrador.resolverSugerencia() valida el permiso y llama a ClubSeneca.resolverSugerencia()	SugerenciaArticulo.aprobar(), SugerenciaArticulo.rechazar()
28	Solicitar un cambio de turno	ClubSeneca.registrarSolicitudTurno()	Empleado.solicitarCambioTurno()
29	Validar el mínimo de 2 entrenadores y 1 fisioterapeuta	ClubSeneca.resolverSolicitudTurno()	GestorTurnos.puedeAprobar(), Turno.cubre()
30	Cambiar el turno al aprobar	ClubSeneca.resolverSolicitudTurno()	SolicitudCambioTurno.aprobar(), Empleado.cambiarTurno()
31	Crear y dar de baja equipos y socios	Administrador.crearEquipo(), Administrador.darDeBajaEquipo(), Administrador.darDeBajaSocio() validan el permiso y llaman a ClubSeneca	ClubSeneca.crearEquipo(), ClubSeneca.crearSocio(), ClubSeneca.darDeBajaEquipo(), ClubSeneca.darDeBajaSocio()
32	Reportes de ventas y deportivos (RF27, RF28)	ClubSeneca.generarReporteVentas(), ClubSeneca.generarReporteDeportivo()	GeneradorReportes
33	Login y password	ClubSeneca.iniciarSesion()	Autenticador.autenticar()

34	Guardar y cargar los datos	ClubSeneca.guardarDatos(), ClubSeneca.cargarDatos()	Persistencia.guardar(), Persistencia.cargar()
35	Crear y mantener los componentes del club	ClubSeneca	no aplica

Justificación de los diagramas de secuencia

Cómo los hicimos

Para los diagramas de secuencia elegimos cinco escenarios. En total quedaron seis diagramas porque dividimos el cambio de turno en dos partes: una para cuando el empleado solicita el cambio y otra para cuando el administrador toma la decisión. Escogimos estos escenarios porque reúnen varias de las reglas del enunciado, involucran movimientos de dinero o puntos y requieren la interacción entre varios objetos.

En todos los diagramas se sigue una estructura similar: la persona le hace una petición a la Consola, que representa el programa de prueba, y esta se comunica con ClubSeneca. Solo en los flujos del administrador (3b y 5) la petición pasa primero por Administrador. En los demás, la Consola llama directo a ClubSeneca, y Empleado o Socio aparecen como objetos a los que ClubSeneca les pide trabajo. Esta clase coordina las operaciones, busca información, hace las llamadas necesarias y guarda los cambios, mientras que las reglas las evalúan las clases que tienen los datos necesarios. ClubSeneca solo hace validaciones simples de entrada (hora, solapamiento, existencias, puntos, compatibilidad de deporte y categoría).

Solo mostramos el flujo en el que todo funciona correctamente, para que los diagramas sean más fáciles de entender. Los casos en los que algo falla los explicamos al lado de cada diagrama, en la sección de “Caminos alternativos”. Tampoco incluimos Persistencia en los diagramas, ya que al terminar cada flujo ClubSeneca llama a Persistencia.guardar(club). Mostrar este paso en todos los diagramas habría generado demasiadas flechas y los habría hecho más difíciles de leer.

- **Pocos objetos:** ningún diagrama tiene más de 8 cajas, contando a la persona y a la Consola.
- **Pocos mensajes pequeños:** utilizamos mensajes que representan operaciones completas.
- **Caminos alternativos:** los explicamos al lado de cada diagrama, en lugar de incluirlos dentro del mismo.

Por último, verificamos que cada mensaje representado en los diagramas corresponda a un método definido en el diagrama de clases.

1. Reservar una mesa de tenis de mesa (H1, RF5, R12)

Escogimos este escenario porque las reservas son una de las operaciones que más realizan los socios y porque permite mostrar varias reglas en un mismo flujo: que la hora de

inicio sea anterior a la final, que la modalidad y el número de jugadores sean válidos, que la instalación no tenga otra reserva en ese horario y que haya cupo disponible en la disciplina.

ClubSeneca verifica que la hora de inicio sea anterior a la final y consulta InstalacionIndividual.admiteModalidad(). Luego obtiene las reservas con Historial.getReservas() y revisa con Reserva.seSolapaCon() que la instalación no tenga otra reserva a esa hora. Después pregunta a DisciplinaIndividual.hayCupo() si no se supera el máximo de instalaciones simultáneas. Si todo se cumple, crea la Reserva y la guarda en el Historial.

2. Comprar en la cafetería con código de empleado y puntos (H4, H7, RF11, RF12, RF13, RF19, R19, R20, R21)

Este es uno de los escenarios más importantes porque involucra el manejo de dinero. En la compra se deben tener en cuenta el impuesto, el descuento para empleados y los puntos de fidelidad. También se aplica la regla de seguridad relacionada con las bebidas calientes.

ClubSeneca coordina el flujo y hace las validaciones simples de entrada. Con Producto.getCantidad() revisa que haya existencias y con Socio.getPuntosFidelidad() que los puntos a redimir alcancen. Si los puntos a redimir son negativos o mayores a los del socio, toma 0. Después le pide la venta a CafeteriaSocios, que verifica con puedeDespachar() si la bebida se puede entregar en la instalación actual (R20, R21). Si se puede, crea la Venta. Venta calcula el total en su constructor con calcularTotal(), que es privado, consultando CodigoDescuento.getPorcentaje() para saber si el descuento es del 15% (el comprador es el empleado dueño del código) o del 8% (no lo es). Con ese total define también los puntos generados.

Con la venta lista, ClubSeneca reduce las existencias con Producto.reducirCantidad(), registra la bebida caliente con Socio.registrarBebidaCaliente() si corresponde, y llama a Socio.redimirPuntos() y Socio.acumularPuntos(). Al final agrega la venta a su lista. Si la compra es en la tienda, se usa Tienda.vender() con IVA del 19% y sin puedeDespachar().

La regla R21 se cumple en este flujo. CafeteriaSocios.puedeDespachar() rechaza una bebida caliente cuando la instalación actual no la permite, usando InstalacionIndividual.permiteBebidaCaliente(). El registro de la hora con registrarBebidaCaliente() deja constancia de cuándo el socio recibió la bebida.

3a. Solicitar el cambio de turno (H5, RF18)

Aunque este flujo es corto, lo incluimos para mostrar quién se encarga de crear la solicitud. ClubSeneca.registrarSolicitudTurno() le pide al Empleado que cree la solicitud con Empleado.solicitarCambioTurno(), que verifica que el turno actual sea uno de sus turnos. Luego ClubSeneca la guarda en su lista. La solicitud queda pendiente hasta que el administrador la revise y tome una decisión.

3b. Resolver la solicitud (H5, H14, RF26, R23)

Escogimos este escenario porque incluye la regla que exige tener siempre un mínimo de 2 entrenadores y 1 fisioterapeuta. Esta validación no podía quedar en una de las clases que

ya teníamos, porque Turno.cubre() únicamente revisa un turno individual. Por eso creamos GestorTurnos, que se encarga de decidir si la solicitud se puede aprobar mediante GestorTurnos.puedeAprobar().

ClubSeneca solo procesa solicitudes que estén en su lista y pendientes. Si el administrador aprueba, ClubSeneca le pasa a GestorTurnos la lista de empleados, que obtiene filtrando sus usuarios. Si se mantiene el mínimo, cambia el estado de la solicitud con SolicitudCambioTurno.aprobar() y actualiza los turnos mediante Empleado.cambiarTurno(). Si no se mantiene el mínimo, se llama a SolicitudCambioTurno.rechazar(). Si el administrador rechaza, también se llama a rechazar(), pero sin consultar a GestorTurnos.

4. Partido interno de fútbol con suspensión por tarjetas (RF16, RF17, R7, R24)

Este escenario es el que involucra más objetos y permite mostrar varias reglas relacionadas con el deporte, especialmente la compatibilidad entre equipos, el mínimo de jugadores habilitados y las suspensiones por tarjetas.

ClubSeneca consulta a cada Equipo con getDeporte() y getCategoria() para verificar que ambos sean del mismo deporte y categoría. Después usa Equipo.puedeJugar() para verificar que cada uno cuente con el mínimo de jugadores habilitados (R24). Si se cumple todo, crea el PartidoInterno, agrega las estadísticas con Partido.agregarEstadistica() y guarda el partido en el Historial.

Las tarjetas son gestionadas por ControlSanciones. Las amarillas y rojas se leen de EstadisticaFutbol, y el jugador se obtiene de EstadisticaConjunto.getJugador(). ClubSeneca llama a InscripcionEquipo.registrarTarjetas(), que le pasa la información a ControlSanciones. Este determina si corresponde suspender al jugador por acumular dos amarillas o recibir una roja.

5. Inscribir un equipo en una competencia (RF29, R9)

Incluimos este escenario porque muestra cómo se relaciona el club con las competencias externas. El administrador inicia la inscripción y ClubSeneca consulta a Competencia con equipoCumpleRequisitos(), que verifica las condiciones de deporte, categoría y nómina mínima de jugadores habilitados. Si se cumplen, ClubSeneca llama a Competencia.inscribirEquipo(), que vuelve a revisar los requisitos y que el equipo no esté ya inscrito, y luego crea la InscripcionCompetencia. Después ClubSeneca agrega la competencia a su lista si faltaba.

La nómina no se arma en este paso, sino aparte. Cada jugador entra con InscripcionCompetencia.agregarANomina(), que crea su RegistroNomina. Esta nómina contiene a los jugadores autorizados para disputar los partidos oficiales, de acuerdo con R9.

Estilo de control: delegado

El diseño sigue un estilo de delegación. ClubSeneca se encarga de coordinar las operaciones, pero no toma las decisiones sobre las reglas del enunciado. Cada escenario tiene una clase responsable de validar sus condiciones: Venta en las compras, GestorTurnos en los cambios de turno, PartidoInterno junto con ControlSanciones en los

partidos, Competencia en las inscripciones y InstalacionIndividual junto con DisciplinaIndividual en las reservas.

Justificación de la implementación de la persistencia

Cómo la hicimos

Con el fin de implementar la persistencia del proyecto, se decidió utilizar las librerías org.json, usadas anteriormente en otros talleres. Mediante estas librerías, se formuló el proceso con el que se trabajaría la persistencia. Este consistía en, mediante un método inicial, comenzar un proceso encadenado en el que se harían llamados a todas las clases y sus atributos, de tal forma que las instancias almacenadas en los atributos que respondan a dichos llamados se transformaran a formato JSON y se almacenaron en el documento de datos. También se ideó que el proceso de guardado se ejecutara al final de cada método que modifica datos, tal como los presentes en ClaseSeneca, de tal forma siempre se almacenaría cualquier modificación y se reduciría el riesgo de perder información o procesos vitales.

El recorrido entre clases

El recorrido de llamadas comienza en la clase principal ClubSeneca, ésta mediante los métodos guardar() y cargar() se encarga de llamar a la clase Persistencia la cual gestionará todo el proceso inicial de llamadas a métodos y atributos. Todos estos métodos que serán ejecutados se organizaron en métodos toJSON() y uno fromJSON para cada clase, siendo respectivamente encargados de transformar lo atributos objetos o String o int o boolean o double a formato JSON, y la otra en leer estos mismo datos a las instancias previamente existentes.

toJSON

En el caso del proceso toJSON, este comienza con el llamado al método toJSON presente en ClubSeneca. Este método se encargará de, para cada atributo que almacene datos ejecutar un llamado, en el caso de los tipo de datos comunes, se transforman inmediatamente a JSON, en el caso de los atributos que contengan objetos, a estos se llamará al método toJSON presente en la clase que lo requiera. En el caso de colecciones de objetos, tipo listas y mapas, se recorrerá la colección entera haciendo un llamado al método toJSON para cada elemento de la colección que sea un objeto o transformando directamente al formato para los tipos comunes.
Al ser un proceso recursivo, este se aplicará mediante el método toJSON de la misma forma como se aplicó en ClubSeneca a cada clase que almacene información. Aun así, en el caso de que un objeto ya haya sido llamado por el método toJSON y este vuelva a ser llamado por otra clase, para evitar duplicidad de objetos e instancias, o errores de bucles infinitos (StackOverflowError) al momento de reconstruir la información, los objetos ya llamados, cuando requieran una segunda instancia, se llamarán únicamente mediante un identificador como el nombre o otros.
Otra condición especial a tratar es la de las herencias, en estas se creará la función toJSON en la superclase que se encargará de llamar todos los datos comunes de las herencias,

después las clases hijas, mediante un @Override y un llamado al super(), se encargarán de especificar el tipo de la instancia y los datos extras necesarios.
Finalmente, después de recibir el resultado tipo JSON de todas las instancias existentes, la clase Persistencia se encarga de crear una carpeta (en caso de que no exista) y crear o modificar el archivo club.json con los datos obtenidos.

fromJSON

En el caso del proceso fromJSON, este comienza con el llamado al método fromJSON presente en ClubSeneca. Este método se encargará de: definir mapas para almacenar y organización de las instancias guardadas, establecer un orden específico al momento de reconstruir el programa para evitar que el programa acceda a objetos inexistentes, reconstruir las instancias mediante métodos fromJSON encadenados que se encarguen de leer las listas del JSON y creen las instancias dadas.
Este proceso funciona en forma similar al guardado respondiendo a cada dato presente en el archivo JSON y creando la respectiva instancia mediante métodos fromJSON presentes en cada clase que lo requiera. Cada uno de estos métodos variará en función de la forma en la que se haya guardado la información como por ejemplo, con las colecciones para las que se tendrá que crear una instancia del constructor para cada conjunto de datos en el archivo. También, relacionado a los mapas, como para cada objeto con el fin de evitar la duplicidad se guardaron ciertos datos mediante identificadores, al momento de reconstruirlo, las primeras clases que transformen los datos a objetos se encargará de guardar la información en mapas para que las clases que requieran los mismos objetos accedan a estas directamente.
En los casos de las herencias, estas se manejan mediante switch en la superclase la cual detectará el tipo de herencia que se guardó previamente y creará la instancia respectiva mediante el método fromJSON de la clase hija respectiva.
Importante añadir que todos los métodos fromJSON deberán ser estáticos para que las clases externas puedan acceder a estos aunque no existan aún instancias de las clases dadas.

Verificación e IA

Todo el proceso de verificación de la persistencia y logia de los métodos, se realizó mediante clases de prueba agrupadas en el paquete de PruebasJson. Estas clases, junto con los datos de prueba respectivos, fueron construidas con IA, con el único fin de verificar los resultados mediante datos simulados para verificar un ciclo válido de cargado y guardado de datos. Los resultados de estas pruebas se utilizaron para pulir código y depurar errores (duplicado de instancias, bucles, etc) mediante IA y para identificar fallas en el código.